State Effects

Design 1: CreatureState Interface with onEnter(), onExit(), checkTransitions() and getAction().

Pros	Cons
Follows SRP as each state classes are solely responsible for its own logic, transitions and entry/exit effects	There are more classes to navigate through therefore can be confusing for other developers viewing the code.
Following OCP as adding a new state does not require any changes to existing states or any actor classes.	
Low connascence of algorithm because each state will implement its own logic independently no state will need to low how another state works (low coupling)	

Design 2: Have all of the logic occur as conditionals inside playTurn() logic for each creature

Pros	Cons
The decision logic is visible in one place	Will violate SRP because the actor class will become responsible for all the state logic.
	Modifying and adding states requires directly editing the actor class therefore this violates OCP.
	Maintenance is difficult because it becomes harder to read as the number of states grows.
	High connascence of algorithm because all state logic depends on each other in one method.

Status Effects

Design 1: Each effect implements the Status interface that includes status such as KineticLinkStatus() and others we will create.

Pros	Cons
Follows SRP as each class is responsible for its own tick logic and the expiration condition.	More classes to navigate.
Actor does not need to manage any behaviour of the effect	
Reusable as the statuses can be applied by any state to any actor without having to copy and paste logic	
Follows KISS each status class is small and focused and easy to understand in isolation.	

Design 2: Have the effects occur directly on the actor and handle them inside playTurn()

Pros	Cons
No need to create new status classes for simple implementations	Actor has the responsibility to manage all of its effects which will become more difficult as more states are added that violate SRP.
	Adding a new effect requires modifying the actor class violating OCP.
	Violates KISS at the method level playTurn() which grows increasingly more complex with each new effect.
	High connascence of algorithm because all effect logic is coupled inside one method.

State Pattern

Design 1 State pattern and StatefulCreature base class.

pros	cons
Following the DRY principle, the state transition loop, DizzyStatus and the initial entry logic is only implemented once and has shared logic across both Meeple and Swarmling.	Added a layer of abstraction, making the class structure deeper
This meets the principle OCP because by adding a state means that you will only have to extend StatefulCreature.	
Satisfy SRP, each state class is responsible only for its own behavior and transition logic	
Low connascence of algorithm because Meeple and Swarmling do not need to know how the state machine loop works internally	

Design 2
Each creature will manage its own state machine independently.

pros	cons
No Shared abstraction required reducing connascence of inheritance.	Violates DRY because the entire state transition loop must be duplicated in every stateful creature class.
	Violates KISS overall because the same complex logic must be repeated and maintained in every creature class.
	Violates SRP because each creature class becomes responsible for its own behaviour and also the state machine loop.
	High connascence of algorithm because any state changes mean that the state machine flow must be replicated across

	every creature class.

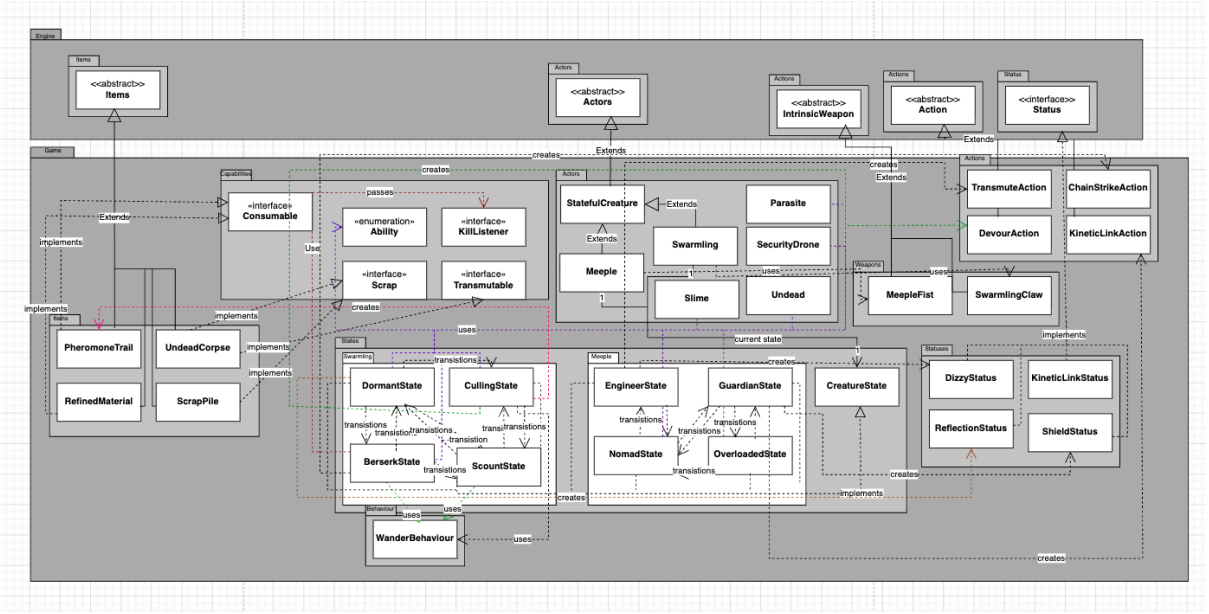
Actor Type

Design 1 Ability Enum capability Checks

pros	cons
Following DIP the state classes will depend on the Ability abstraction and not on concrete actor classes.	Requires manual calling of enableAbility() in each new actor's constructor.
Following OCP the new actor type will automatically be detected by all existing states without needing additional modification.	
State classes do not need to import concrete actor classes, therefore eliminating tight coupling between packages.	

Design 2: Using instanceof to identify actors within the logic

Pros	Cons
Obvious which class is being targeted	Will violate DIP because every state must import and depend on the concrete actor classes.
	Also violates OCP because adding a new actor requires updating every state that detects it.
	Causes tight coupling between the states package and the actor package



Shown in the UML our requirement 5 decision was to implement two stateful creatures which we choose: the Meeple and Swarmling. Each has four distinct states that will transition deterministically based on environmental conditions such as Worker HP levels, kill counts and parasite proximity. Each state will produce an immediate entry effect and new behaviours will be introduced through custom actions and status which will all be decoupled from concrete actor classes via the Ability enum and the CreatureState interface.

We chose design 1 for all four decisions. This includes for the “state structure” and “state pattern” design decision we decided to implement a CreatureState interface which has a composition relationship with the StatefulCreature abstract class. This is important because the interface will act as a level of abstraction between the StatefulCreature and the concrete state classes and remove direct dependencies between the creature classes and any specific state implementation (DIP). This also meets OCP because if new states are introduced in the future no modification is required to StatEfulCreature or any other existing state classes, rather a new state simply implements CreatureState. Each state class is solely responsible for its own behaviour, transition logic and entry/exit effects because they are distributed amongst the different state classes rather than being concentrated in playTurn() (SRP). By separating each state into its own class, connascence of algorithm principle is low as there is no state class required to know how another states logic works internal, therefore reducing the overall coupling of the system.

We also choose design 1 for the Ability enum as this acts as a level of abstraction between the state classes and the actor classes. State classes including the CullingState,

DormantState and EngineerState all depend on the Ability enum rather than on the concrete actor classes such as Parasite and Undead. Therefore, this removes tight coupling between actors and states packaging (DIP). This also prevents the downcasting of instanceof checks to determine the actor types. This means that when new enemy types are introduced no state class needs to be modified, rather the new actor class calls enableAbility() in its constructor to automatically be recognised by all existing states (OCP).

The design decision for “Status Effects” chooses design 1 because the Status interface follows the same pattern with DizzyStatus, KineticLinkStatus, ReflectionStatus and ShieldStatus which each implement Status, ensuring that each effect is responsible for its own tick logic and for their expiry condition rather than burdening the actor class with managing all effects directly (SRP).